

Train with Ramps, Deploy with Ranks: Learnable Weighted Order Statistic Cascades as Tiny, Robust, Multiplication-Free Alternatives to Neural Networks

Juan Carlos del Río*

October 2026 (preprint)

Abstract

Order-statistic filters are increasing, 1-Lipschitz in ℓ_∞ , have explicit breakdown points and need no multiplications, but they are hard to train because a selection is piecewise constant in its parameters. We parameterise cascades of flat weighted order statistic (WOS) filters by real repetition weights and a quantile, train them through a *ramp* relaxation that spreads each sample’s mass between its rank neighbours, which gives exact, non-vanishing gradients without temperatures, and deploy the *exact* WOS cascade, so every guarantee holds for the deployed model. When a linear read-out follows the selections, annealing the ramp to the hard selection during training is necessary. We benchmark the resulting models (WOS-R) against residual convolutional neural networks (CNNs), multilayer perceptrons (MLPs) and small Transformers on five problems, reporting accuracy, operator coverage and cost. With 16–137 parameters and at most 8 multiplications per sample, WOS-R comes within 1.2 dB of a 22k-parameter CNN on 1D impulsive restoration and beats it when the impulse amplitude doubles, where a Transformer breaks down. On the BSD68 natural-image test set, a switching WOS-R with 52 parameters is 0.5 dB behind a 75k-parameter denoising CNN (DnCNN) at 50 % salt-and-pepper noise, and plain WOS-R cascades are 5.6–6.7 dB ahead of it on random-valued impulses never seen in training. In radar constant-false-alarm-rate (CFAR) detection, a 34-parameter WOS-R is the best detector in the presence of interfering targets, where an MLP loses up to 4.3 dB and a Transformer never reaches a detection probability $P_d = 0.5$. In federated aggregation, robustness comes from rank cascades, not from learned aggregators. The coverage study delineates the class: at 35 dB signal-to-noise ratio (SNR) the deployed WOS-R recovers median, erosion and weighted-WOS operators exactly, where all networks remain 5–27 dB above the noise floor; it fails by construction on signed linear and level-dependent operators, and ramp training does not always find representable two-stage cascades.

1 Introduction

Rank-order (order-statistic) filters—median, erosion/dilation, weighted medians, stack filters—were a mainstay of nonlinear signal processing in the 1980s and 1990s because they remove impulses while preserving edges, are translation invariant and need no multiplications [1–3]. A recurring difficulty was *design*: superposition does not apply, and early adaptive schemes relied on least-mean-squares (LMS) stochastic gradients through implicit formulations of the rank operation [4–6]. In 1993 the author studied the adaptation of *cascades* of two order filters [7], a problem that is harder than adapting a single filter because the composition is not reducible to one filter, as it is in the linear case. That work showed that structures whose parameters act only on the *index* of the selected sample (a binary mask and a rank) cannot be adapted in cascade, and that structures in which parameters enter the *value* of the selected sample can.

*Independent researcher, Madrid, Spain. jcdelrio100@gmail.com. Preprint DOI: [10.5281/zenodo.PPDOI](https://doi.org/10.5281/zenodo.PPDOI). Abbreviations are expanded at first use and listed in Appendix A.

Three decades later, deep networks dominate signal and image restoration, and the questions have changed: how *small*, *robust* and *cheap* can a learned model be, and what guarantees does it keep? Order-statistic models are attractive in this regard. They are Lipschitz by construction, they have explicit breakdown points, they can be implemented with comparators only, and their parameters are interpretable. Their weakness remains trainability: a selection operator is piecewise constant in its parameters, so gradients vanish almost everywhere. This is the same “gradient sparsity” recently identified for morphological networks [8].

This paper. We revisit that problem with modern tools and propose a simple recipe, *train with ramps, deploy with ranks* (WOS-R):

1. We parameterise each stage as a flat weighted order statistic (WOS) filter, i.e. a filter in which each sample of the window enters the sorted list with a real-valued *repetition count* $w_j \geq 0$, together with a quantile r (Section 3). This is the classical weighted median/WOS family [3, 9, 10], and we make its expressiveness precise for cascades.
2. During training only, we replace each repetition by a *ramp* that spreads the sample’s mass between the mid-points to its rank neighbours (Section 4). The resulting quantile is continuous and piecewise linear in (w, r) , so exact gradients exist everywhere without temperatures, straight-through estimators [11] or soft sorting [12–14]. At deployment the ramp is removed and the model is an *exact* cascade of selections, so all the guarantees of rank filters apply to the model that is actually used.
3. We benchmark WOS-R against residual convolutional neural networks (CNNs, in the style of the denoising CNN DnCNN [15]), multilayer perceptrons (MLPs) and small Transformers [16] on five problems: 1D impulsive restoration, an operator-coverage matrix (system identification), image impulse denoising on the standard Set12 and BSD68 (Berkeley segmentation dataset, 68 images) test sets [17], radar constant-false-alarm-rate (CFAR) detection in K-distributed sea clutter [18, 19], and Byzantine/fault-robust aggregation in federated learning (FL; training distributed over nodes, some of them faulty or malicious) [20–22]. We report accuracy, *coverage* (which operator classes each model can and cannot represent) and cost: parameters, multiplications and comparisons per output sample, and CPU latency.

Contributions. (i) A training recipe for exact WOS cascades: repetition weights plus a ramp relaxation used only during training, with ramp continuation when a linear read-out follows (Sections 3–4). (ii) Properties of the deployed model: integer equivalence, class- $\mathcal{O}$ Lipschitz bound independent of depth, breakdown point as a function of the repetitions, and a Matheron–Maragos test of when a cascade collapses to one WOS. (iii) A benchmark of accuracy, coverage and cost against CNN, MLP and Transformer baselines on five problems, with all code and result files (Section 6). (iv) A delimitation of where order-statistic models are preferable—impulsive and heavy-tailed contamination, out-of-distribution amplitudes and contamination types, interference-limited detection, Byzantine aggregation, and multiplication-free hardware—and where they are not—Gaussian noise in distribution, signed linear operators and contamination rates beyond the design breakdown point (Section 7).

2 Related work

Order-statistic and stack filters. Weighted median and WOS filters [3, 9] are stack filters [2] whose Boolean function is a threshold function [10]. Arce’s weighted median admits negative weights [23]. Adaptive design used LMS-type or sigmoidal approximations [3–6] and, later, support vector machines (SVMs) on the threshold decomposition [24]. Salembier’s adaptive rank-order filters [4] and their cascades [7] are the direct ancestors of this work.

Morphological and tropical networks. Morphological perceptrons [25, 26], smooth morphological layers [27], sparse hybrid linear–morphological networks [28] and the tropical view of ReLU networks [29] relate max-plus algebra to deep learning. Training difficulties are attributed to gradient sparsity and initialisation sensitivity [8]. Sorting-based activations also yield Lipschitz networks [30].

Differentiable sorting. NeuralSort [12], projection-based soft sort/rank [13] and differentiable sorting networks [14] relax *permutations*. Our ramp relaxes the *quantile of a weighted empirical distribution*, is exact at deployment ($\beta = 0$), and leaves the sorting itself hard.

Robust aggregation and CFAR. Coordinate-wise median and trimmed mean [20], Krum [31], centred clipping [21] and nearest-neighbour mixing [22] are order-statistic aggregators facing attacks such as “a little is enough” (ALIE, small coordinated shifts hidden within the honest spread) [32] and inner-product manipulation (IPM, updates that reverse the sign of the honest mean’s inner product) [33]. Order-statistic CFAR (OS-CFAR, threshold from the k -th ranked reference cell) [18] and its greatest-of/smallest-of (GO/SO) variants [34] are rank-filter detectors.

3 Weighted order statistic cascades

3.1 Definitions

Let $z \in \mathbb{R}^m$ be the window of a signal or image around the current sample, $w \in \mathbb{R}_{\geq 0}^m$ the repetition weights, $W = \sum_j w_j$ and $r \in (0, 1)$. The WOS output is the lower weighted r -quantile

$$\text{WOS}_{w,r}(z) = \min \left\{ z_{(k)} : \sum_{j: z_j \leq z_{(k)}} w_j \geq rW \right\}. \quad (1)$$

For $w \in \mathbb{N}^m$ this is exactly “sort the list in which z_j appears w_j times and take position $\lceil rW \rceil$ ”. $w_j = 0$ removes the tap (a mask); $w \equiv 1$ gives the rank filters, and $r \rightarrow 0$, $r \rightarrow 1$ give erosion and dilation. A WOS-R model is a *cascade* of WOS stages, or a *bank* of C cascades combined by an affine mix $y = \sum_c a_c y_c + b$. The mix contributes the only C multiplications per output sample.

3.2 Properties

Proposition 1 (repetitions, thresholds and integers). (i) $(\lambda w, r)$ and (w, r) define the same filter for all $\lambda > 0$. (ii) By threshold decomposition, $\text{WOS}_{w,r}(z) \geq t$ iff $\sum_{j: z_j \geq t} w_j > (1-r)W$: a positive threshold Boolean function. (iii) Every real-weight WOS equals an integer-weight WOS, because there are finitely many threshold functions of m variables.

Proposition 2 (class $\mathcal{O}$ and Lipschitz). Every flat WOS cascade (and every cascade of ordered weighted averages, OWA $\sum_k w_k z_{(k)}$, including the 1993 NEST structure $\sum_k |h_k|(z + b)_{(k)}$, with weights in the simplex) is increasing, commutes with translations and satisfies $\Psi(z + c) = \Psi(z) + c$. Consequently $\|\Psi(x) - \Psi(y)\|_\infty \leq \|x - y\|_\infty$, independently of depth.

Proof. If $x \leq y + \delta$, then $\Psi x \leq \Psi(y + \delta) = \Psi y + \delta$; exchange x and y . $\square$

Proposition 3 (breakdown). A set S of arbitrarily corrupted window samples can drive $\text{WOS}_{w,r}$ to $\pm\infty$ iff $w(S) \geq \min\{\lceil rW \rceil, W - \lceil rW \rceil + 1\}$. Boosting the weight of a position therefore reduces the number of corrupted samples needed: for a 5-point median with centre weight 3, two samples suffice instead of three.

Proposition 4 (Matheron–Maragos basis). Every flat cascade is $\Psi z = \max_{B \in \mathcal{B}} \min_{b \in B} z_b$, where $\mathcal{B}$ is the set of minimal true sets of its Boolean function [35, 36]. A rank- r filter of m samples needs $\binom{m}{m-r+1}$ erosions, against $m+1$ WOS parameters. A cascade is a single WOS iff its Boolean function is a threshold function; a mixed-integer linear program on the truth table decides this and returns the minimal integer repetitions.

Table 1: Measured properties with non-uniform weights $w = (1, 2, 3, 1, 0.5, 2, 1)$, $r = 0.4$, on $2 \cdot 10^4$ random windows of 7 samples. “Jump in r ”: largest output change when r moves by $5 \cdot 10^{-3}$. Last two columns: largest output magnitude of the 5-point (ramp) median with uniform weights under k impulses of 10^6 .

Operator	Increasing	$\Psi(z+c)=\Psi(z)+c$	Lipschitz ℓ_∞	Jump in r	2 impulses	3 impulses
Exact WOS ($\beta = 0$)	yes	yes	1.00	3.46	3.9	$1.0 \cdot 10^6$
Ramp, $\beta = 1$	no	yes	5.73	0.17	$2.5 \cdot 10^5$	$7.5 \cdot 10^5$
Ramp, $\beta = 0.5$	no	yes	3.88	1.78	—	—
Ramp, integer version	no	yes	9.14	—	3.9	$1.0 \cdot 10^6$

On the 20 two-stage cascades used in our experiments, 8 are single WOS (verified with zero error), while openings, median-of-medians and the cascade studied in 1993 are not. This motivates cascades and banks.

4 The ramp relaxation

Definition 5 (ramp WOS). Sort z and let $m_k^- = z_{(k)} - \beta \frac{z_{(k)} - z_{(k-1)}}{2}$ and $m_k^+ = z_{(k)} + \beta \frac{z_{(k+1)} - z_{(k)}}{2}$ (extremes are not extended), with $\beta \in [0, 1]$. Sample k carries mass $w_{(k)}$ spread uniformly over $[m_k^-, m_k^+]$. The ramp-WOS output is the r -quantile of this piecewise-linear distribution:

$$y = m_{k^*}^- + \frac{rW - C_{k^*-1}}{w_{(k^*)}} (m_{k^*}^+ - m_{k^*}^-), \quad k^* = \min\{k : C_k \geq rW\}, \quad C_k = \sum_{i \leq k} w_{(i)}. \quad (2)$$

The integer version places $w_{(k)}$ equispaced points in $[m_k^-, m_k^+]$; for example $(0, 1), (0.5, 3), (1, 1)$ yields the list $[0, 0.25, 0.5, 0.75, 1]$.

Proposition 6. (i) $\beta = 0$ recovers (1) exactly. (ii) For $\beta = 1$ the cells tile $[z_{(1)}, z_{(m)}]$, so y is continuous and piecewise linear in (w, r) , with $\partial y / \partial r$ and $\partial y / \partial w$ defined almost everywhere and non-zero on a set of positive measure. (iii) With uniform weights and $r = \frac{1}{2}$ (odd m), the ramp median is the L -filter $\frac{1}{4}z_{(k-1)} + \frac{1}{2}z_{(k)} + \frac{1}{4}z_{(k+1)}$. (iv) With non-uniform weights, the ramp is not continuous in z : swapping two samples with different weights re-assigns cells. It is neither increasing nor 1-Lipschitz, and its breakdown point drops by one rank neighbour.

Property (iv) is why we use the ramp *only for training*. The deployed model ($\beta = 0$) is an exact WOS cascade and satisfies Propositions 2–4. Table 1 quantifies (ii) and (iv).

Remark 7 (ramp continuation for linear read-outs). For a pure cascade, the ramp’s interpolation error is a common-mode bias that the discrete parameters cannot absorb, and the deployed selection snaps to the right order statistic: in our coverage experiment the ramp-trained chain has an excess error of 18.5–21.6 dB at $\beta = 1$ and 0.0 dB *after deployment* on every single-WOS target (Table 3). When a linear layer follows the selections (the affine mix of a bank), the mix can co-adapt to the ramp, combining slightly different quantiles into an interpolation that disappears at $\beta = 0$; a ramp-only bank then misses even a 7-point median by 20 dB. Annealing β linearly from 1 to 0 during training (we use the first 70 % of the steps) removes the effect; refitting the mix by least squares after training does not, because the chains themselves have co-adapted. We therefore train chains with $\beta = 1$ and banks with this ramp continuation.

Why it helps. With the exact WOS, $\partial y / \partial w = \partial y / \partial r = 0$ almost everywhere. The straight-through alternative uses the gradient of a temperature-smoothed cumulative distribution function (CDF) evaluated at the hard output; the soft alternative anneals a temperature. The ramp provides the exact gradient of a nearby continuous model whose distance to the deployed WOS is bounded by half the local spacing of the order statistics.

5 Experimental protocol

All experiments run on a 2-core CPU with PyTorch. WOS-R stages are trained with Adam ($\eta = 0.03$, cosine schedule) at $\beta = 1$ (banks: β annealed $1 \rightarrow 0$ over the first 70 % of the steps, Remark 7) and evaluated at $\beta = 0$; baselines use Adam ($\eta = 10^{-3}$ or $2 \cdot 10^{-3}$) with the same data, loss (mean squared error, MSE) and number of steps. “Mults” counts multiply–accumulates per output sample (attention included), “Comps” counts comparisons for sorting (bound $m \lceil \log_2 m \rceil$ per stage), and latency is single-threaded CPU time. Exact configurations are listed in Appendix C; the code and all result files are available at https://github.com/GITHUB_USER/wosr-rank-filters (archived at [doi:10.5281/zenodo.SWDOI](https://doi.org/10.5281/zenodo.SWDOI)).

6 Results

6.1 B1: one-dimensional restoration

Table 2 reports restoration of piecewise-smooth 1D signals, measured by the normalised mean squared error ($\text{NMSE} = \|\hat{s} - s\|^2 / \|s\|^2$ in dB; lower is better). Under impulsive noise, a two-stage WOS-R chain with 16 parameters and no multiplications reaches -20.5 dB, better than the 3857-parameter medium CNN (CNN-M; S/M/L denote small/medium/large) (-19.4 dB), the MLP (-19.9 dB) and the Transformer (-18.5 dB). These three runs are, however, favourable for the single chain: over 20 independent trainings (B1, the convergence study and the restart study in the released code) it reaches -18.7 ± 1.3 dB (range -20.7 to -16.9), and keeping the best of 4 restarts selected on validation data gives -19.6 dB, on a par with CNN-M. Banks are far less sensitive to initialisation (-21.1 ± 0.5 dB over 6 runs for $C=8$). The bank with $C=8$ chains (137 parameters, 8 multiplications per sample) reaches -21.1 dB, 1.2 dB behind the best model, CNN-L, which uses 161 times more parameters and 2744 times more multiplications. When the impulse amplitude doubles (out of distribution, OOD $\times 2$), the ordinal models degrade least: WOS-R bank $C=8$ (-17.8 dB) and the 1993 NEst bank (-18.2 dB) overtake CNN-L (-17.4 dB), while the Transformer breaks down ($+0.3$ dB, against $+9.7$ dB for the noisy input), as Proposition 2 would predict for a model that is not 1-Lipschitz. When the impulse *rate* increases to 25 %, the CNNs extrapolate better (-15.5 vs -12.4 dB): a WOS cascade designed for 10 % contamination has a fixed breakdown point (Prop. 3).

Under Gaussian noise, averaging is required and CNNs win in distribution (-20.4 vs -18.2 dB); at twice the noise level the ranking reverses and WOS-R bank $C=8$ is the best model (-12.7 dB vs -11.6 dB for CNN-L). Ramp continuation improves the banks by 0.9–1.5 dB over ramp-only training (ablation rows), consistently with Remark 7. Keeping the ramp at test time ($\beta = 1$) is 0.9–1.3 dB *worse* under impulses in distribution (0.3–1.4 dB over all impulsive columns) but 1.0 dB better for the Gaussian chain, because the $\beta = 1$ ramp median is an L-filter (Prop. 6(iii)); we keep $\beta = 0$ throughout for the guarantees.

6.2 B2: coverage

Table 3 probes coverage by system identification at high SNR (35 dB), where approximate emulation is penalised. Each cell is the excess error of the deployed model over the noise floor of the true operator; Fig. 1 shows the same data as a map. Four facts stand out.

(i) *Exactness*. When the ramp-trained WOS-R finds the target, it finds it *exactly*: 0.0 dB on the 7-point median, the flat erosion, the weighted WOS and rank cascade #1, for both seeds and both architectures. No network reaches the floor on any rank-type target: the MLP, CNN-M and Transformer stay 5.5–26.8 dB above it, i.e. they approximate sorting but do not represent it, with 56–328 times more parameters.

(ii) *Optimisation, not representation, is the bottleneck inside the class*. The opening and rank cascades #12 and #5 are two-stage rank cascades, hence exactly representable by the WOS-R chain, yet ramp training does not find them (12–25 dB). This agrees with the identification

Table 2: B1: 1D restoration, normalised mean squared error (NMSE) in dB (mean $\pm$ std over 3 seeds; lower is better). Impulsive models are trained with 10 % impulses; OOD (out-of-distribution) columns change impulse rate or amplitude. NEst: 1993 thesis structure; CNN-S/M/L: small/medium/large residual CNN. Gaussian models are trained with $\sigma = 0.2$. Params, multiply-accumulates and comparisons per output sample; ms: CPU latency for 4096 samples (1 thread). Best per column in bold.

Model	Params	Mults	Comps	ms	in-dist	Impulsive		Gaussian	
						OOD $p=25\%$	OOD $\times 2$	$\sigma=0.2$	$\sigma=0.4$ (OOD)
Median 5	0	0	15	0.3	-13.7 ± 1.2	-3.6 ± 0.3	-8.0 ± 1.3	-14.9 ± 0.3	-8.9 ± 0.4
WOS-R chain 2×7	16	0	42	2.1	-20.5 ± 0.1	-11.3 ± 0.3	-16.5 ± 1.7	-16.4 ± 0.3	-11.1 ± 0.4
WOS-R bank $C=4$	69	4	168	7.8	-20.8 ± 0.5	-12.1 ± 0.5	-17.4 ± 1.1	-17.9 ± 0.2	-12.4 ± 0.4
WOS-R bank $C=8$	137	8	336	16.9	-21.1 ± 0.6	-12.4 ± 0.6	-17.8 ± 0.9	-18.2 ± 0.4	-12.7 ± 0.4
ablation: bank $C=4$, ramp only	69	4	168	7.8	-19.9 ± 0.9	-11.1 ± 1.6	-16.1 ± 1.4	—	—
ablation: bank $C=8$, ramp only	137	8	336	16.9	-19.8 ± 0.9	-10.9 ± 1.7	-16.5 ± 1.2	—	—
NEst bank $C=4$	117	60	168	3.7	-21.0 ± 0.4	-12.3 ± 0.3	-18.2 ± 0.9	-18.5 ± 0.2	-12.5 ± 0.3
CNN-S	129	120	0	0.1	-15.0 ± 0.7	-8.3 ± 0.2	-9.4 ± 0.9	-19.1 ± 0.3	-11.3 ± 0.3
CNN-M	3 857	3 808	0	0.7	-19.4 ± 0.4	-12.9 ± 1.0	-14.6 ± 0.6	-20.4 ± 0.3	-12.1 ± 0.6
CNN-L	22 081	21 952	0	3.8	-22.3 ± 0.6	-15.5 ± 0.7	-17.4 ± 0.0	-20.4 ± 0.2	-11.6 ± 0.7
MLP (window 15)	5 249	5 120	0	1.5	-19.9 ± 0.3	-12.9 ± 0.5	-15.0 ± 0.1	-19.7 ± 0.3	-12.5 ± 0.4
Transformer	4 561	69 728	0	141.0	-18.5 ± 0.3	-11.1 ± 0.2	0.3 ± 0.3	-16.5 ± 0.3	-9.6 ± 0.4

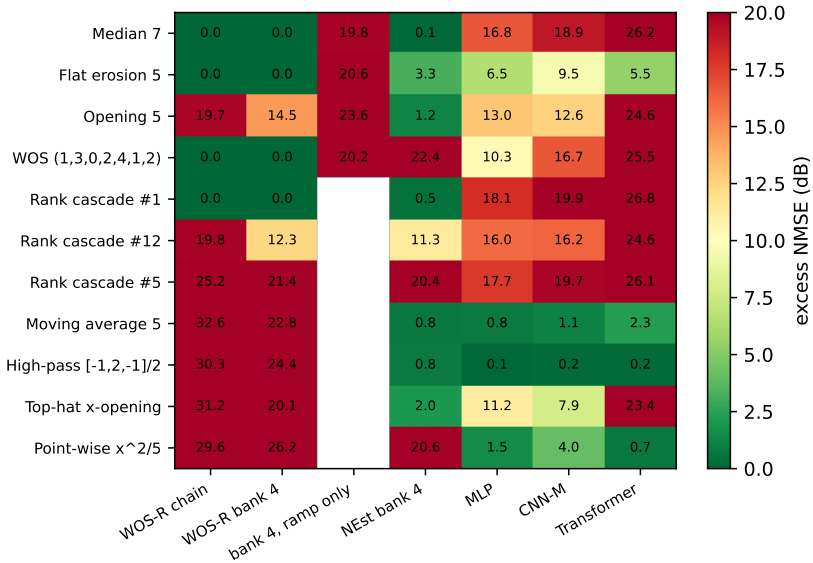


Figure 1: Coverage map (excess NMSE above the floor, dB; green: represented, red: not represented).

rate of 9/16 random cascades reported earlier. The 1993 NEst bank, whose parameters act on values, finds the opening (1.2 dB) and cascade #1 but misses the weighted WOS (22.4 dB): the two parameterisations fail on different targets.

(iii) *Outside the class.* A flat WOS cascade is increasing and commutes with level shifts (Prop. 2), so it cannot represent the signed high-pass, the point-wise square or the top-hat; the bank’s affine mix could in principle express the top-hat as identity minus opening, but inherits the failure to find the opening. Moving averages are not selections either; the bank approaches them only coarsely (22.8 dB). Networks represent these operators well (0.1–2.3 dB for the linear ones).

(iv) *The deployed model, not the relaxed one.* At $\beta = 1$ the same trained models are 17–31 dB above the floor on every target; the exact selections appear only at $\beta = 0$. Ramp training is a means to find the discrete model, not an approximation used at inference.

6.3 B3: image impulse denoising

Table 5 reports blind impulse denoising on BSD68 (Set12 in Appendix B). All learned models are trained on salt-and-pepper (SP) noise with random density in $[0.1, 0.6]$; random-valued impulses (RVIN) are never seen in training. Three regimes appear.

Quality is measured by the peak signal-to-noise ratio (PSNR, dB) and the structural similarity

Table 3: B2 coverage: excess normalised mean squared error (NMSE, dB) above the noise floor of the true operator (input $U[-5, 5]$, SNR 35 dB, mean of 2 seeds). Bold: ≤ 1 dB (operator represented); red: > 6 dB (not represented). Parameters in the last row.

Target	Class	WOS-R chain	WOS-R bank 4	bank 4, ramp only	NEst bank 4	MLP	CNN-M	Transformer
Median 7	rank	0.0	0.0	19.8	0.1	16.8	18.9	26.2
Flat erosion 5	morphological	0.0	0.0	20.6	3.3	6.5	9.5	5.5
Opening 5	morph., 2 stages	19.7	14.5	23.6	1.2	13.0	12.6	24.6
WOS (1, 3, 0, 2, 4, 1, 2)	WOS	0.0	0.0	20.2	22.4	10.3	16.7	25.5
Rank cascade #1	rank cascade	0.0	0.0	–	0.5	18.1	19.9	26.8
Rank cascade #12	rank cascade	19.8	12.3	–	11.3	16.0	16.2	24.6
Rank cascade #5	rank cascade	25.2	21.4	–	20.4	17.7	19.7	26.1
Moving average 5	linear, $h \geq 0$	32.6	22.8	–	0.8	0.8	1.1	2.3
High-pass $[-1, 2, -1]/2$	linear, signed	30.3	24.4	–	0.8	0.1	0.2	0.2
Top-hat x -opening	non-increasing	31.2	20.1	–	2.0	11.2	7.9	23.4
Point-wise $x^2/5$	not translation-inv.	29.6	26.2	–	20.6	1.5	4.0	0.7
Params		16	69	69	117	5 249	3 857	4 561

Table 4: B3 coverage check: Gaussian denoising (trained at $\sigma = 25$), PSNR in dB, Set12 / BSD68. Order statistics are not the right tool here (Prop. 2: no averaging).

Model	Params	$\sigma = 25$	$\sigma = 50$ (OOD)
Median 3×3	0	25.45 / 24.89	21.06 / 20.92
WOS-R bank $C=4$ (3×3)	85	26.64 / 26.15	21.95 / 21.84
WOS-R bank $C=4$ (5×5)	213	27.00 / 26.45	22.56 / 22.42
MLP 5×5	5 889	28.19 / 27.59	19.88 / 20.00
DnCNN-S (6 layers, 16 ch)	9 585	29.00 / 28.24	18.64 / 18.79
DnCNN-M (10 layers, 32 ch)	74 593	29.36 / 28.51	17.62 / 17.81
Window Transformer	17 697	28.15 / 27.58	20.04 / 20.08

index (SSIM, 0–1). *In distribution*, the best model is DnCNN-M, the 10-layer DnCNN (74 593 parameters, 74 304 multiplications per pixel). A *switching* WOS-R, which applies a learned 5×5 two-stage WOS cascade only to pixels that are an extreme of their 3×3 neighbourhood, reaches 26.22 dB at SP 50 % (0.52 dB behind DnCNN-M) with 52 parameters and no multiplications, has a higher SSIM (0.840 vs 0.761), matches the 17-layer DnCNN (26.20 dB) and beats the window Transformer (24.92 dB). The hand-designed adaptive median filter (AMF) remains a remarkably strong baseline for SP noise (26.02 dB at 50 %, best SSIM at 70 %); the switching WOS-R matches it between 30 % and 50 % and is 1.6–1.8 dB behind at the extremes of the density range.

Out of distribution (RVIN), every model that relies on detecting extreme values collapses: AMF, the switching WOS-R, the MLP, the DnCNNs and the Transformer all fall to 16–19.5 dB at RVIN 40 %. The plain (non-switching) WOS-R cascades keep 23.4–24.5 dB, *5.6–6.7 dB above DnCNN-M*, because a selection with a bounded breakdown point does not care whether an outlier is at 0, 1 or anywhere in between. Figure 2 shows both effects.

Gaussian noise (Table 4) is outside the natural class of order statistics: on BSD68, WOS-R banks are 2.1–2.4 dB behind DnCNN-M at the training level $\sigma = 25$, but 4.0–4.6 dB *ahead* at $\sigma = 50$ (Set12: 2.4–2.7 and 4.3–4.9 dB), where every convolutional model over-fits its training noise level. In 2D, ramp continuation and ramp-only training give banks within 0.7 dB of each other, without a consistent sign; the co-adaptation of Remark 7 matters at high SNR (B2) and in 1D (B1).

DnCNN-17 without batch normalisation did not train in our CPU budget (17.06 dB on Set12 at SP 10 %); we report the original design with batch normalisation, which after 2 000 steps is still below the 10-layer DnCNN-M. Our DnCNNs are therefore *under-trained* relative to the published model [15], and the in-distribution gaps above are lower bounds on what a GPU-trained network achieves; the OOD collapses are not an artefact of under-training, since they affect every network regardless of its in-distribution quality.

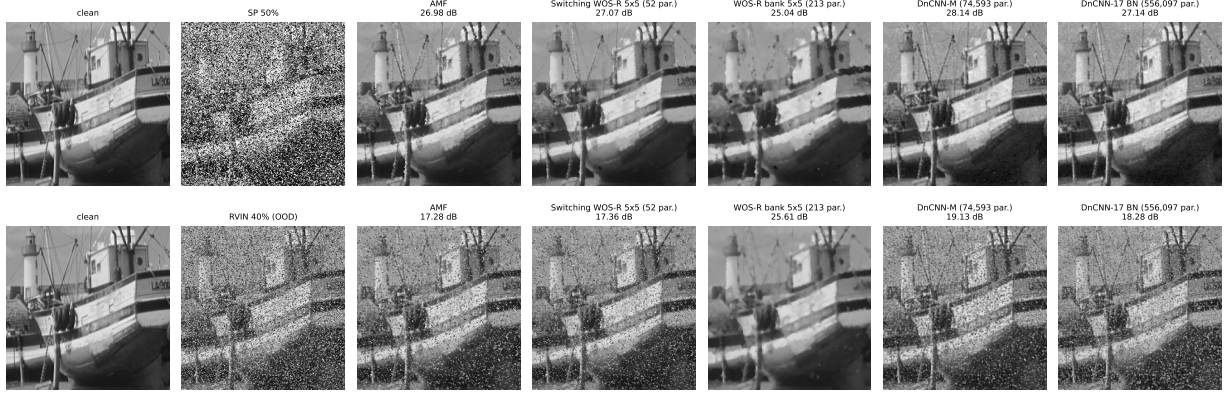


Figure 2: Set12 “boat” (256×256 crop; PSNR of the full image). Top: SP 50 % (in distribution). Bottom: RVIN 40 % (never seen in training). The switching WOS-R matches AMF and approaches DnCNN-M on SP; the plain (non-switching) WOS-R removes random-valued impulses, which every detection-based model and every network misses.

Table 5: B3: blind impulse denoising on BSD68, peak signal-to-noise ratio (PSNR, dB; higher is better) / structural similarity (SSIM). Training: salt-and-pepper (SP) with density $\sim U[0.1, 0.6]$. Out of distribution (OOD): SP 70 %, random-valued impulse noise (RVIN). AMF: adaptive median filter; switching: WOS-R applied only to pixels that are an extreme of their 3×3 window. Mults and comparisons per pixel; latency in ms for a 512×512 image (1 CPU thread). Best per column in bold.

Model	Params	Mults	Comps	ms	SP 10%	SP 30%	SP 50%	SP 70%	RVIN 20%	RVIN 40%
Median 3×3	0	0	36	52	28.26/0.851	21.94/0.682	14.62/0.268	9.58/0.069	26.70 /0.802	21.62/0.552
Median 5×5	0	0	125	149	25.79/0.731	24.90/0.711	21.49/0.617	13.48/0.226	25.36/0.718	23.59/0.637
Adaptive median (AMF, 7)	0	0	var.	386	32.29/0.944	29.07/0.909	26.02/0.839	20.06/0.647	21.05/0.524	16.07/0.260
WOS-R 3×3, 2 stages	20	0	72	178	27.37/0.812	25.50/0.774	19.25/0.562	11.90/0.154	26.63/0.793	23.73/0.668
WOS-R 5×5, 2 stages	52	0	250	722	26.65/0.771	25.83/0.751	23.77/0.705	15.83/0.422	26.19/0.757	24.46 /0.696
WOS-R bank $C=4$ (3×3)	85	4	288	489	26.46/0.785	25.41/0.755	20.47/0.535	13.37/0.162	25.78/0.770	23.38/0.661
WOS-R bank $C=4$ (5×5)	213	4	1000	2806	26.60/0.770	25.89/0.752	24.11/0.699	16.85/0.372	26.07/0.756	24.37/0.696
ablation: bank $C=4$ (3×3), ramp only	85	4	288	489	26.98/0.801	25.62/0.766	20.22/0.532	13.11/0.157	26.22/0.784	23.55/0.667
ablation: bank $C=4$ (5×5), ramp only	213	4	1000	2806	26.74/0.776	25.98/0.757	24.08/0.704	16.55/0.369	26.20/0.762	24.43/0.700
Switching WOS-R 3×3	20	0	90	240	31.18/0.933	27.88/0.894	20.19/0.657	12.53/0.197	20.98/0.513	16.17/0.257
Switching WOS-R 5×5	52	0	268	666	30.52/0.923	28.88/0.902	26.22/0.840	18.48/0.604	20.89/0.505	16.19/0.253
Switching WOS-R bank $C=4$ (3×3)	85	4	306	726	29.78/0.885	27.76/0.843	21.87/0.602	14.82/0.218	20.82/0.501	16.11/0.253
MLP 5×5	5889	5760	0	129	30.09/0.892	24.37/0.642	19.42/0.384	15.44/0.200	21.87/0.513	17.98/0.317
DnCNN-S (6 layers, 16 ch)	9585	9504	0	44	29.27/0.868	26.36/0.734	23.64/0.578	19.94/0.375	23.66/0.628	19.43/0.424
DnCNN-M (10 layers, 32 ch)	74593	74304	0	616	33.65 /0.950	29.97 /0.881	26.74 /0.761	21.92 /0.521	22.08/0.530	17.74/0.336
DnCNN-17 (BN, 64 ch)	556097	554112	0	3065	31.41/0.911	29.21/0.835	26.20/0.713	21.74/0.502	20.96/0.463	16.45/0.280
Window Transformer	17697	25152	0	1247	30.37/0.915	28.01/0.834	24.92/0.681	20.17/0.437	20.28/0.447	16.80/0.291

6.4 B4: CFAR detection

Table 6 compares detectors that estimate the local clutter level from the same 32 reference range cells around the cell under test (CUT), in the log domain, and are calibrated to the same false-alarm probability $P_{fa} = 10^{-4}$; the figure of merit is the SNR needed for a detection probability $P_d = 0.5$ (SNR₅₀). The learned WOS-R (32 repetition weights, a quantile and a scale: 34 parameters) is the best detector whenever interfering targets are present, at both texture correlations: with 8 interferers it gains 1.5 dB ($\rho = 0$) and 1.7 dB ($\rho = 0.9$) over OS-CFAR. In homogeneous clutter the log-equivariant MLP (6337 parameters; “log-equivariant” means that scaling the clutter scales the threshold by the same factor, as CFAR requires) is 0.3 dB better, but it pays 4.3 dB ($\rho = 0$) and 3.3 dB ($\rho = 0.9$) with 8 interferers: a smooth function of the reference cells is contaminated by the interferers that an order statistic ignores up to its breakdown point. The Transformer, trained with the same Neyman–Pearson objective, is competitive without interferers and never reaches $P_d = 0.5$ up to 30 dB SNR when two or more interferers are present. The learned repetitions are interpretable (Fig. 3): with uncorrelated texture they stay nearly uniform (an OS-CFAR with a learned rank), whereas with $\rho = 0.9$ the cells adjacent to the guard cells receive 3–4 times the weight of distant cells, the positional boosting that a rank filter cannot express.

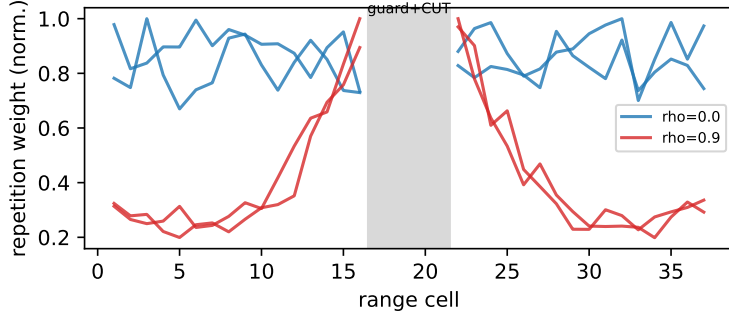


Figure 3: Learned repetition weights of the CFAR WOS-R (2 seeds per texture correlation), normalised to their maximum.

Table 6: B4: constant-false-alarm-rate (CFAR) detection in K-distributed clutter with range-correlated texture (AR(1) coefficient ρ), false-alarm probability $P_{fa} = 10^{-4}$ calibrated per detector. Signal-to-noise ratio (SNR, dB) for detection probability $P_d = 0.5$ with 0/2/4/8 interfering targets in the reference window (lower is better); learned detectors: mean of 2 seeds, trained with the same Neyman–Pearson objective.

Detector	Params	$\rho = 0$				$\rho = 0.9$			
		0	2	4	8	0	2	4	8
OS-CFAR ($k=24$)	0	15.46	16.41	17.33	19.91	14.43	15.30	16.27	18.75
GO-OS ($k=11$)	0	15.76	16.63	17.50	20.09	14.91	15.72	16.50	18.85
WOS-R (32 repetitions)	34	15.55	16.29	16.95	18.38	14.20	14.83	15.53	17.09
MLP (log-equivariant)	6 337	15.22	17.64	19.37	22.69	13.86	15.92	17.57	20.35
Transformer (log-equivariant)	5 009	15.55	> 30	> 30	> 30	14.85	> 30	> 30	> 30

6.5 B5: robust aggregation

Table 7 evaluates aggregation rules in federated training with 4 of 20 faulty nodes. No single-shot aggregator survives all attacks: burst faults that persist over consecutive steps already drive the mean and both trimmed means to chance accuracy (0.10), and the combined Gaussian-plus-burst fault does the same to the coordinate-wise (CW) median, the adaptive minimax OWA (AMO, an OWA whose weights are chosen online from a precomputed minimax library) and the learned sorted-MLP aggregator; only centred clipping keeps a little above chance (0.14–0.18). A learned MLP on sorted inputs standardised by the median and the median absolute deviation (MAD) (5 569 parameters, trained on synthetic attack families) is no exception: silent data corruption (SDC: sporadic huge values from faulty hardware), which is not among its training families, drives it to 0.11, since nothing bounds the influence of an input that leaves the training range. What restores robustness is the *cascade* structure of the thesis applied to the time $\times$ node axes: a per-node temporal median over the last 3 submissions, followed by nearest-neighbour mixing and a final order-statistic aggregator. This ordinal cascade, with no trained parameters, has the best worst-case accuracy for $\alpha = 1$ (0.753). Replacing its last stage by the learned MLP improves the heterogeneous worst case ($\alpha = 0.1$: 0.615 vs 0.535, driven by IPM) at a small cost in the homogeneous case (0.740). The robustness comes from the rank cascade; the learned stage adds accuracy only once the cascade has bounded the influence of the faulty inputs.

6.6 Cost summary

Table 8 pairs the best WOS-R model with the best network on each task. Parameter savings are $161\times$ (1D), $186\times$ (CFAR) and $1\,434\times$ (images); multiplications drop from $6\cdot 10^3$ – $7\cdot 10^4$ per sample to 0–8, replaced by a few hundred comparisons. The accuracy cost in distribution is 0.5–1.2 dB; out of distribution the sign flips. Wall-clock latency with generic PyTorch kernels does *not* reflect these savings: sorting unfolded windows is memory-bound and the bank evaluates C chains sequentially,

Table 7: B5: Byzantine/fault-robust aggregation (Fashion-MNIST, $n=20$ nodes, $f=4$ faulty, 300 steps, mean of 2 seeds). Test accuracy under each attack with heterogeneous data (Dirichlet $\alpha=0.1$), and worst case over attacks for $\alpha \in \{1, 0.1\}$. CW: coordinate-wise; Med_t3: per-node temporal median of the last 3 submissions; NNM: nearest-neighbour mixing [22]; AMO: adaptive minimax ordered weighted average (OWA). Attacks: ALIE (“a little is enough”), IPM (inner-product manipulation), burst-8 (faults persisting 8 steps), SDC (silent data corruption). [†]AMO uses a precomputed library of 5 OWA weight vectors (minimax second-order cone program), no training.

Aggregator	Params	$\alpha = 0.1$							worst case	
		none	sign-flip	ALIE	IPM	burst-8	SDC	Gauss+burst	$\alpha=1$	$\alpha=0.1$
Mean	0	0.814	0.414	0.787	0.789	0.100	0.100	0.100	0.100	0.100
CW median	0	0.603	0.599	0.565	0.465	0.244	0.683	0.100	0.100	0.100
CW trimmed mean	0	0.728	0.642	0.728	0.372	0.100	0.707	0.100	0.100	0.100
NNM + CW trim	0	0.793	0.727	0.714	0.695	0.100	0.803	0.100	0.100	0.100
Centred clipping	0	0.806	0.681	0.771	0.693	0.707	0.803	0.141	0.177	0.141
AMO (minimax OWA)	0 [†]	0.591	0.551	0.610	0.410	0.247	0.602	0.100	0.100	0.100
Med _t 3 → AMO	0 [†]	0.576	0.502	0.527	0.264	0.542	0.548	0.539	0.691	0.264
Med _t 3 → NNM → CW trim	0	0.779	0.672	0.772	0.535	0.782	0.773	0.762	0.753	0.535
Sorted-MLP aggregator	5 569	0.673	0.646	0.718	0.380	0.101	0.113	0.100	0.100	0.100
Med _t 3 → NNM → sorted-MLP	5 569	0.777	0.703	0.767	0.615	0.780	0.775	0.773	0.740	0.615

Table 8: Cost summary: best WOS-R vs best network per task. Mults/Comps per output sample or pixel; ms: CPU latency (1 thread; 4096 samples in 1D, 512×512 in 2D) with generic PyTorch kernels.

Task	WOS-R model	Params	Mults	Comps	ms	Score	Network	Params	Mults	ms	Score
B1 impulsive (NMSE dB)	WOS-R bank $C=8$	137	8	336	16.9	-21.1	CNN-L	22 081	21 952	3.8	-22.3
B1 OOD amplitude (NMSE dB)	WOS-R bank $C=8$	137	8	336	16.9	-17.8	CNN-L	22 081	21 952	3.8	-17.4
B3 BSD68 SP 50% (PSNR)	Switching WOS-R 5×5	52	0	268	665.9	26.22	DnCNN-M (10 layers, 32 ch)	74 593	74 304	616.2	26.74
B3 BSD68 RVIN 40% (PSNR)	WOS-R 5×5, 2 stages	52	0	250	722.0	24.46	DnCNN-M (10 layers, 32 ch)	74 593	74 304	616.2	17.74
B4 CFAR, $\rho=0.9$, 8 interf. (SNR ₅₀ dB)	WOS-R	34	1	160	–	17.09	MLP	6 337	6 208	–	20.35

so WOS-R is 1.1–4.5 times slower than CNNs of similar accuracy on a CPU. The operation counts, not this latency, are what a comparator implementation would exploit.

7 Discussion

What the benchmark says. Across the five problems the pattern is consistent. WOS-R is competitive in distribution with networks two to four orders of magnitude larger whenever the contamination is impulsive or heavy-tailed, and it is *better* whenever the test distribution moves in a direction that a selection ignores: larger impulse amplitudes (B1), a different impulse type (B3, RVIN), or a higher Gaussian level (B1, B3); the same mechanism makes it the best detector in the presence of interfering targets (B4), which were present in training but contaminate any smooth function of the reference cells. These are the directions in which a 1-Lipschitz, bounded-influence model cannot be surprised, whereas networks trained by empirical risk minimisation have no such guarantee; the Transformer shows the most severe failures (B1, B4). Conversely, WOS-R loses when the task requires averaging in distribution (Gaussian noise, ≈ 2 dB), when the contamination rate exceeds the design breakdown point (B1, $p = 25\%$, ≈ 3 dB), and, by construction, on the operators outside the class $\mathcal{O}$ (B2): signed linear filters, non-increasing operators such as the top-hat, and point-wise nonlinearities. Table 9 summarises the trade-off.

Cascades as robust front-ends. The federated experiment suggests the most practical use: a rank cascade first bounds the influence of corrupted inputs, and a learned stage then adds accuracy (B5: 0.535→0.615 worst-case accuracy under heterogeneity). The same composition applies to images (a WOS-R front-end followed by a small CNN) and to radar (WOS-R CFAR followed by a learned classifier). Because a WOS cascade is increasing and translation invariant, any back-end that is 1-Lipschitz in ℓ_∞ preserves the end-to-end bound.

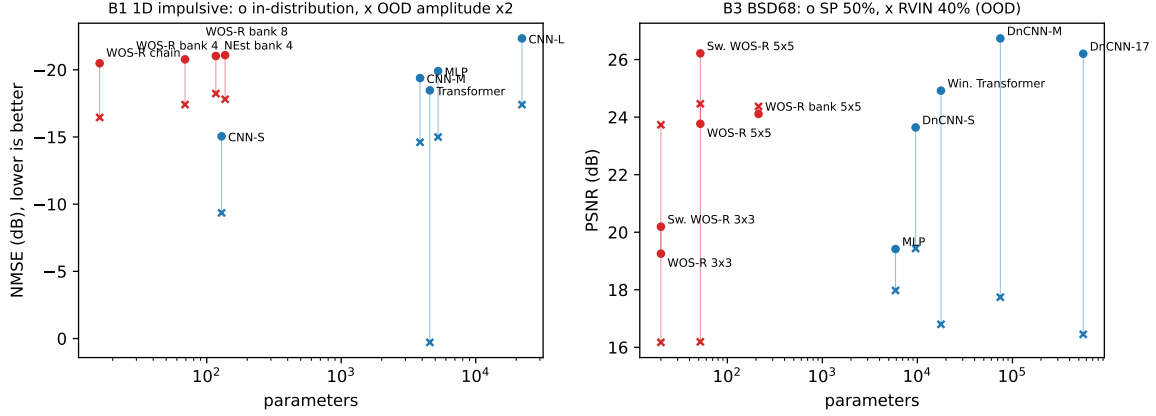


Figure 4: Accuracy versus parameters (log scale). Red: order-statistic models; blue: networks. Vertical segments join in-distribution (o) and out-of-distribution (x) scores of the same model: short segments mean robust models.

Table 9: When to use WOS-R (summary of B1–B5).

WOS-R preferable	Network preferable
Impulsive/heavy-tailed noise, unknown amplitude or type	Gaussian noise at the training level
Interference-limited detection (CFAR)	Contamination rate above the design breakdown
Byzantine/fault-robust aggregation (as a cascade front-end)	Signed linear, non-increasing or level-dependent operators
≤ 200 parameters, ≤ 8 multiplications, certified Lipschitz/breakdown	Highest in-distribution PSNR with GPU training

The 1993 question, revisited. The thesis [7] concluded that parameters acting only on the *index* of the selected sample (masks, ranks) cannot be adapted in a cascade, while parameters entering the *value* can. Repetition weights are index parameters, and exact WOS stages indeed have zero gradients almost everywhere. The ramp resolves the dilemma by giving index parameters a value effect *only during training*: the gradient flows through the interpolation between rank neighbours, and the interpolation is removed at deployment. In the identification task of the thesis, this raises the fraction of random cascades identified from 2–3 of 16 (NEst, value parameters) to 9 of 16 (paired Wilcoxon $p = 4.6 \cdot 10^{-8}$ over 3 seeds).

Hardware. A deployed WOS-R stage needs a sorting network and a running sum of integer repetitions (Prop. 1(iii)); a bank adds C multiplications per sample. This maps to comparator-only logic on field-programmable gate arrays (FPGAs) and low-power digital signal processors (DSPs), the setting in which stack and weighted-median filters were originally implemented [2, 3].

8 Limitations

All training ran on a 2-core CPU. Image models use a single training seed and at most 3 000 steps, so the CNN and Transformer baselines are under-trained relative to published GPU models; we mitigate this by reporting their failure modes only where they persist across model sizes. The CFAR study uses simulated K-distributed clutter with an AR(1) texture model, and the federated study a small CNN on Fashion-MNIST [37] with 20 nodes. Latency is measured with generic PyTorch kernels (sorting by `torch.sort` on unfolded windows), which favour convolutions; the multiplication and comparison counts are the hardware-relevant quantities, and a sorting-network or comparator implementation remains to be built. Finally, the ramp gives exact gradients of a

relaxed model but no guarantee of finding the global optimum: a two-stage chain represents the opening exactly, yet ramp training does not find it (Table 3), and random cascades are identified in 9 of 16 cases.

9 Conclusion

We have shown that weighted order statistic cascades, a 40-year-old family of nonlinear filters, become trainable with a simple device—a ramp relaxation used only during training—and that the resulting models keep their guarantees at deployment. With tens to a few hundred parameters and almost no multiplications, they match or approach CNNs, MLPs and Transformers that are two to four orders of magnitude larger on impulsive restoration and detection, and order-statistic models are the most robust in every out-of-distribution test that changes the amplitude, the impulse type or the noise level—though not when the contamination rate exceeds their design breakdown point. Their limits are equally clear: they are not universal approximators and should be combined with linear or learned stages when the target is outside the class of increasing, translation-invariant operators. Learned switching detectors, hybrid WOS–linear layers and comparator hardware are natural next steps.

Historical note. The structures studied in [7] (rank-order filters (ROF), LL-filters and the NEst structure $y = \sum_k |h_k|(z + b)_{(k)}$) were adapted sample by sample with LMS and implicit-function derivatives. Re-implemented with exact reverse-mode gradients, the 1993 NEst cascade reaches the noise floor on the thesis’ own example in 7 s instead of 20 s, but identifies only 2–3 of 16 random cascades. Repetition weights with ramp training identify 9 of 16.

References

- [1] Ioannis Pitas and Anastasios N. Venetsanopoulos. *Nonlinear Digital Filters: Principles and Applications*. Kluwer Academic Publishers, 1990.
- [2] Peter D. Wendt, Edward J. Coyle, and Neal C. Gallagher. Stack filters. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 34(4):898–911, 1986.
- [3] Lin Yin, Ruikang Yang, Moncef Gabbouj, and Yrjö Neuvo. Weighted median filters: a tutorial. *IEEE Transactions on Circuits and Systems II*, 43(3):157–192, 1996.
- [4] Philippe Salembier. Adaptive rank order based filters. *Signal Processing*, 27(1):1–25, 1992.
- [5] Ioannis Pitas and S. Vougioukas. LMS order statistic filters adaptation by back propagation. *Signal Processing*, 25(3):319–335, 1991.
- [6] Michaël Ropert and Danielle Pelé. Synthesis of adaptive weighted order statistic filters with gradient algorithms. In *Mathematical Morphology and Its Applications to Image Processing*. Springer (Kluwer), Dordrecht, 1994.
- [7] Juan Carlos del Río Romero. Adaptación de estructuras y filtros de orden (adaptation of structures and order filters). Master’s thesis, Escola Tècnica Superior d’Enginyers de Telecomunicació de Barcelona, Universitat Politècnica de Catalunya, 1993. Supervisor: P. Salembier.
- [8] Mihaela Dimitrova, Samy Blusseau, and Santiago Velasco-Forero. Optimizing morphological representations: robustness to initialization and gradient sparsity. *Journal of Mathematical Imaging and Vision*, 2026.
- [9] D. R. K. Brownrigg. The weighted median filter. *Communications of the ACM*, 27(8):807–818, 1984.

- [10] Olli Yli-Harja, Jaakko Astola, and Yrjö Neuvo. Analysis of the properties of median and weighted median filters using threshold logic and stack filter representation. *IEEE Transactions on Signal Processing*, 39(2):395–410, 1991.
- [11] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. In *arXiv:1308.3432*, 2013.
- [12] Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations. In *Proc. ICLR*, 2019.
- [13] Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking. In *Proc. ICML*, 2020.
- [14] Felix Petersen, Christian Borgelt, Hilde Kuehne, and Oliver Deussen. Differentiable sorting networks for scalable sorting and ranking supervision. In *Proc. ICML*, 2021.
- [15] Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang. Beyond a Gaussian denoiser: residual learning of deep CNN for image denoising. *IEEE Transactions on Image Processing*, 26(7):3142–3155, 2017.
- [16] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: hierarchical vision transformer using shifted windows. In *Proc. IEEE/CVF ICCV*, 2021.
- [17] David Martin, Charless Fowlkes, Doron Tal, and Jitendra Malik. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proc. ICCV*, 2001.
- [18] Hermann Rohling. Radar CFAR thresholding in clutter and multiple target situations. *IEEE Transactions on Aerospace and Electronic Systems*, 19(4):608–621, 1983.
- [19] Keith D. Ward, Robert J. A. Tough, and Simon Watts. *Sea Clutter: Scattering, the K Distribution and Radar Performance*. IET, 2 edition, 2013.
- [20] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: towards optimal statistical rates. In *Proc. ICML*, 2018.
- [21] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Learning from history for Byzantine robust optimization. In *Proc. ICML*, 2021.
- [22] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafaël Pinot, and John Stephan. Fixing by mixing: a recipe for optimal Byzantine ML under heterogeneity. In *Proc. AISTATS*, 2023.
- [23] Gonzalo R. Arce. A general weighted median filter structure admitting negative weights. *IEEE Transactions on Signal Processing*, 46(12):3195–3205, 1998.
- [24] Chih-Chia Yao and Pao-Ta Yu. The optimal design of weighted order statistics filters by using support vector machines. *EURASIP Journal on Advances in Signal Processing*, 2006:024185, 2006.
- [25] Gerhard X. Ritter and Peter Sussner. An introduction to morphological neural networks. In *Proc. 13th International Conference on Pattern Recognition (ICPR)*, 1996.
- [26] Vasileios Charisopoulos and Petros Maragos. Morphological perceptrons: geometry and training algorithms. In *Proc. ISMM, LNCS 10225*, 2017.

- [27] Romain Hermay, Guillaume Tochon, Élodie Puybareau, Alexandre Kirszenberg, and Jesús Angulo. Learning grayscale mathematical morphology with smooth morphological layers. *Journal of Mathematical Imaging and Vision*, 64:736–753, 2022.
- [28] Konstantinos Fotopoulos, Christos Garoufis, and Petros Maragos. Sparse hybrid linear-morphological networks. *arXiv:2504.09289*, 2025.
- [29] Liwen Zhang, Gregory Naitzat, and Lek-Heng Lim. Tropical geometry of deep neural networks. In *Proc. ICML*, 2018.
- [30] Cem Anil, James Lucas, and Roger Grosse. Sorting out Lipschitz function approximation. In *Proc. ICML*, 2019.
- [31] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Proc. NeurIPS*, 2017.
- [32] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: circumventing defenses for distributed learning. In *Proc. NeurIPS*, 2019.
- [33] Cong Xie, Oluwasanmi Koyejo, and Indranil Gupta. Fall of empires: breaking Byzantine-tolerant SGD by inner product manipulation. In *Proc. UAI*, 2019.
- [34] Prashant P. Gandhi and Saleem A. Kassam. Analysis of CFAR processors in nonhomogeneous background. *IEEE Transactions on Aerospace and Electronic Systems*, 24(4):427–445, 1988.
- [35] Georges Matheron. *Random Sets and Integral Geometry*. Wiley, 1975.
- [36] Petros Maragos. A representation theory for morphological image and signal processing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(6):586–599, 1989.
- [37] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*, 2017.
- [38] Harish Dattatraya Dixit et al. Silent data corruptions at scale. *arXiv:2102.11245*, 2021.
- [39] Humor Hwang and Richard A. Haddad. Adaptive median filters: new algorithms and results. *IEEE Transactions on Image Processing*, 4(4):499–502, 1995.

A Abbreviations

ALIE	“a little is enough” attack	MAD	median absolute deviation
AMF	adaptive median filter	Med ₃	per-node temporal median of the last 3 updates
AMO	adaptive minimax OWA aggregator	MLP	multilayer perceptron
BN	batch normalisation	MSE	mean squared error
BSD68	Berkeley segmentation dataset, 68 test images	N _{Est}	1993 rank structure $\sum_k h_k (z + b)_{(k)}$
CClip	centred clipping	NMSE	normalised mean squared error
CDF	cumulative distribution function	NNM	nearest-neighbour mixing
CFAR	constant false-alarm rate	OOD	out of distribution
CNN	convolutional neural network	OS-CFAR	order-statistic CFAR
CUT	cell under test	OWA	ordered weighted average
CW	coordinate-wise	P_d, P_{fa}	detection / false-alarm probability
DnCNN	denoising CNN [15]	PSNR	peak signal-to-noise ratio
DSP	digital signal processor	ROF	rank-order filter
FFN	feed-forward network	RVIN	random-valued impulse noise
FL	federated learning	SDC	silent data corruption
FPGA	field-programmable gate array	SNR, SNR ₅₀	signal-to-noise ratio (at $P_d = 0.5$)
GO/SO-OS	greatest-of/smallest-of OS-CFAR	SP	salt-and-pepper noise
IPM	inner-product manipulation attack	SSIM	structural similarity index
K-distr.	compound-Gaussian sea-clutter model	STE	straight-through estimator
LMS	least mean squares	SVM	support vector machine
Mults/Comps	multiplications/comparisons per output	WOS	weighted order statistic
		WOS-R	WOS trained with ramps, deployed exact

Table 10: B3: blind impulse denoising on Set12, peak signal-to-noise ratio (PSNR, dB; higher is better) / structural similarity (SSIM). Training: salt-and-pepper (SP) with density $\sim U[0.1, 0.6]$. Out of distribution (OOD): SP 70%, random-valued impulse noise (RVIN). AMF: adaptive median filter; switching: WOS-R applied only to pixels that are an extreme of their 3×3 window. Mults and comparisons per pixel; latency in ms for a 512×512 image (1 CPU thread). Best per column in bold.

Model	Params	Mults	Comps	ms	SP 10%	SP 30%	SP 50%	SP 70%	RVIN 20%	RVIN 40%
Median 3×3	0	0	36	52	29.02/0.888	22.31/0.713	14.87/0.279	9.74/0.076	27.40 /0.848	22.41/0.614
Median 5×5	0	0	125	149	26.38/0.809	25.05/0.785	21.63/0.682	13.63/0.245	25.77/0.795	23.95/0.727
Adaptive median (AMF, 7)	0	0	var.	386	33.09/0.957	29.15/0.925	25.87/0.864	20.11/0.674	21.63/0.541	16.59/0.273
WOS-R 3×3 , 2 stages	20	0	72	178	28.07/0.868	25.73/0.829	19.39/0.598	11.92/0.163	27.14/0.851	24.35/0.742
WOS-R 5×5 , 2 stages	52	0	250	722	27.44/0.842	26.19/0.820	23.79/0.770	15.86/0.453	26.78/0.829	24.85 /0.777
WOS-R bank $C=4$ (3×3)	85	4	288	489	27.13/0.847	25.77/0.815	20.70/0.576	13.47/0.176	26.31/0.833	24.02/0.740
WOS-R bank $C=4$ (5×5)	213	4	1000	2806	27.31/0.840	26.23/0.821	24.15/0.764	16.82/0.403	26.61/0.827	24.74/0.777
ablation: bank $C=4$ (3×3), ramp only	85	4	288	489	27.79/0.860	25.98/0.822	20.42/0.571	13.19/0.170	26.84/0.845	24.24/0.744
ablation: bank $C=4$ (5×5), ramp only	213	4	1000	2806	27.48/0.845	26.33/0.825	24.12/0.768	16.56/0.398	26.75/0.832	24.80/0.779
Switching WOS-R 3×3	20	0	90	240	32.01/0.951	27.88/0.912	20.19/0.670	12.49/0.198	21.61/0.538	16.68/0.272
Switching WOS-R 5×5	52	0	268	666	31.27/0.943	28.99/0.924	26.03/0.870	18.43/0.629	21.54/0.532	16.69/0.269
Switching WOS-R bank $C=4$ (3×3)	85	4	306	726	30.86/0.907	28.17/0.865	22.10/0.620	14.89/0.226	21.47/0.526	16.64/0.269
MLP 5×5	5889	5760	0	129	30.41/0.880	24.56/0.637	19.69/0.396	15.64/0.214	21.84/0.492	18.11/0.314
DnCNN-S (6 layers, 16 ch)	9585	9504	0	44	29.57/0.878	26.49/0.744	23.82/0.600	20.23/0.417	23.87/0.622	19.66/0.428
DnCNN-M (10 layers, 32 ch)	74593	74304	0	616	33.64 /0.949	30.10 /0.883	26.87 /0.766	22.00 /0.536	22.38/0.520	18.26/0.338
DnCNN-17 (BN, 64 ch)	556097	554112	0	3065	29.95/0.882	28.95/0.812	26.11/0.704	21.41/0.505	21.50/0.470	17.37/0.300
Window Transformer	17697	25152	0	1247	30.23/0.914	27.98/0.834	25.05/0.701	20.72/0.474	20.77/0.450	17.40/0.300

B Additional tables

C Hyper-parameters

- **WOS-R**: repetition weights $w = \text{softplus}(\alpha)$, initialised at ≈ 1 ; quantile $r = \sigma(\rho)$, initialised at 0.5; training at $\beta = 1$ for chains and with β annealed linearly from 1 to 0 over the first 70% of the steps for banks, deployment at $\beta = 0$; Adam, $\eta = 0.03$, cosine decay to 5%. 1D: windows of 7 samples, 2 stages per chain, banks of $C \in \{1, 4, 8\}$ chains with affine mix. 2D: 3×3 or 5×5 windows, 2 stages, $C \in \{1, 4\}$.
- **1D baselines**: residual CNN-S (3 layers, 4 channels, kernel 5), CNN-M (4 layers, 16 channels, kernel 7), CNN-L (5 layers, 32 channels, kernel 7); MLP on a 15-sample window ($15 \rightarrow 64 \rightarrow 64 \rightarrow 1$, residual); Transformer (conv embedding to $d=16$, 2 encoder layers, 2 heads, feed-forward network (FFN) width 32, full attention, residual). Adam, $\eta = 2 \cdot 10^{-3}$, 3000 steps, batch 32 crops of 256 samples.
- **2D WOS-R**: chains of 2 stages on 3×3 or 5×5 windows; banks of 4 such chains; switching variants apply the model only where the centre pixel equals the minimum or maximum of its 3×3 window (18 extra comparisons per pixel) and pass the input through elsewhere.
- **2D baselines**: DnCNN-type residual networks, 3×3 convolutions: S (6 layers, 16 channels) and M (10 layers, 32 channels) without batch normalisation; DnCNN-17 (17 layers, 64 channels, batch normalisation, the original design [15]; the variant without batch normalisation did not train); MLP on a 5×5 neighbourhood; window Transformer (conv embedding to $d=32$, 2 blocks of 8×8 window attention, the second shifted, 4 heads, FFN 64). Adam, $\eta = 10^{-3}$, 3000 steps (2000 for DnCNN-17 and WOS-R), batches of 16 patches of 48×48 from the 400 BSD training images, with random rotations and flips. A single training seed per model (CPU budget).
- **B2 coverage**: input i.i.d. $U[-5, 5]$, 64 sequences of 2048 samples, additive uniform noise at 35 dB SNR; 3000 steps, batches of 16 crops of 256 samples; metric: NMSE of the deployed model minus the NMSE of the true operator applied to the noisy input (the floor). Models: WOS-R chain (2×7), WOS-R bank $C=4$, NEst bank $C=4$, MLP (window 15), CNN-M, Transformer, as in B1.
- **B4 CFAR**: 32 reference cells (16 leading, 16 lagging), 2 guard cells per side; K-distributed clutter with shape $\nu = 2$, scene power uniform in ± 20 dB, and Gaussian-copula AR(1) texture with coefficient $\rho \in \{0, 0.9\}$ along range; detectors act on log reference cells and output

a log-threshold; training: 800 Adam steps of the Neyman–Pearson Lagrangian (sigmoid detection surrogate, temperature 0.25, quadratic penalty on $\log P_{fa}$ above 10^{-4}), with 0/2/4/8 interferers (20 dB above the local clutter) and target SNR uniform in 10–20 dB; evaluation: threshold scale calibrated on $4 \cdot 10^5$ clutter-only cells, P_d on 10^5 cells per SNR on a 5 dB grid from 5 to 30 dB, SNR at $P_d = 0.5$ by linear interpolation. MLP: 32→64→64→1 on mean-removed logs; Transformer: $d=16$, 2 layers, 2 heads, learned positions, mean pooling.

- **B5 aggregation:** Fashion-MNIST, small CNN, 20 nodes of which 4 faulty, Dirichlet label split $\alpha \in \{1, 0.1\}$, worker momentum 0.9, 300 steps, learning rate 0.08; attacks: sign-flip, ALIE [32], IPM [33], burst faults on 8 consecutive steps, SDC [38], Gaussian plus burst. Sorted-MLP aggregator: per coordinate, sort the n values, standardise by median and MAD, MLP 20→64→64→1 (5 569 parameters), trained on synthetic attack families to output the honest mean.
- **Classical:** median, adaptive median filter (Hwang–Haddad, maximum window 7) [39], OS-CFAR, GO-OS, coordinate-wise median and trimmed mean.